

GridLens — Core Source Code

Computational engine (model · parser · VA solver) · EE 374 Term Project · 4 files

Contents

- `src/gridlens/core/models.py`
- `src/gridlens/core/parser.py`
- `src/gridlens/core/solver.py`
- `src/gridlens/core/__init__.py`

src/gridlens/core/models.py

```
1  """Network domain model.
2
3  Pure dataclasses – no Qt imports. Solver and parser operate on these types.
4  """
5  from __future__ import annotations
6
7  from dataclasses import dataclass, field
8
9
10 @dataclass
11 class Bus:
12     id: str
13     name: str = ""
14     base_kv: float = 1.0
15     is_slack: bool = False
16     is_leaf: bool = False
17     v_set_pu: float | None = None # operator-pinned magnitude (leaf override)
18
19
20 @dataclass
21 class Line:
22     id: str
23     from_bus: str
24     to_bus: str
25     r_pu: float = 0.0
26     x_pu: float = 0.0
27     b_pu: float = 0.0
28     rating_a: float | None = None
29     # A transformer is modelled as a branch too (it joins two buses and carries a
30     # series reactance). When ``is_transformer`` is set, ``r_pu``/``x_pu`` already
31     # hold the impedance on the *system* base so the solver treats it like any
32     # other branch; the ``xfmr_*`` fields keep the nameplate data for round-trip
33     # save back to the course file format. ``from_bus`` is the HV side.
34     is_transformer: bool = False
35     xfmrRatedMVA: float | None = None
36     xfmr_hv_kv: float | None = None
37     xfmr_lv_kv: float | None = None
38     xfmr_x_pu: float | None = None # reactance on the transformer's own MVA base
39
40
41 @dataclass
42 class Load:
43     id: str
44     bus: str
45     p_kw: float = 0.0
46     q_kvar: float = 0.0
47     s_rated_mva: float = 0.0 # nameplate apparent power (operating P/Q set by hand)
48
49
50 @dataclass
51 class Generator:
52     id: str
53     bus: str
54     p_kw: float = 0.0
55     q_kvar: float = 0.0
56     s_rated_mva: float = 0.0 # nameplate apparent power (operating P/Q set by hand)
57
58
59 @dataclass
60 class Capacitor:
61     id: str
62     bus: str
63     q_kvar: float = 0.0
64     in_service: bool = True
65
66
67 @dataclass
68 class Network:
69     name: str = ""
70     base_mva: float = 1.0
```

```

71     buses: list[Bus] = field(default_factory=list)
72     lines: list[Line] = field(default_factory=list)
73     loads: list[Load] = field(default_factory=list)
74     generators: list[Generator] = field(default_factory=list)
75     capacitors: list[Capacitor] = field(default_factory=list)
76
77
78 @dataclass
79 class BusSolution:
80     bus_id: str
81     v_pu: float
82     angle_deg: float
83     violation: str = "ok" # "ok" | "under" | "over"
84
85
86 @dataclass
87 class SweepStep:
88     """One inner backward-forward sweep iteration, tagged with the outer
89     Q-compensation pass it belongs to.
90
91     ``pinned_err`` is the worst pinned-bus voltage error ( $|v_{set} - |V||$ ) measured
92     at the *end* of an outer pass; it is None for inner iterations that are not a
93     pass boundary and for networks without voltage-controlled (pinned) buses.
94     """
95
96     outer: int
97     inner: int
98     max_dv: float
99     pinned_err: float | None = None
100
101
102 @dataclass
103 class SolutionResult:
104     converged: bool
105     iterations: int
106     max_mismatch: float
107     bus_results: list[BusSolution] = field(default_factory=list)
108     message: str = ""
109     mismatch_history: list[float] = field(default_factory=list)
110     # Full solve trajectory across every outer Q-compensation pass (the table on
111     # the Solver page renders this); ``mismatch_history`` above stays as the inner
112     # history of the final pass for backwards compatibility.
113     steps: list[SweepStep] = field(default_factory=list)
114     outer_iterations: int = 1
115     # Reactive power (kvar) the outer loop had to inject at each pinned bus to
116     # hold its target  $|V|$  – i.e. the reactive support that sizing a capacitor /
117     # SVC there would need to provide. Empty when no bus is voltage-pinned.
118     pinned_q_inject_kvar: dict[str, float] = field(default_factory=dict)

```

src/gridlens/core/parser.py

```
1  """Network file parser.
2
3  The on-disk format is the course-provided JSON schema (see ``data/FORMAT.md``):
4  ``system_data`` / ``bus_data`` / ``load_data`` / ``gen_data`` / ``shunt_data`` /
5  ``line_data`` / ``transformer_data``. :func:`load_network` reads it into the
6  internal :class:`~gridlens.core.models.Network` model (the solver and UI operate
7  on that model, unaffected by the file shape) and :func:`save_network` writes it
8  back out in the same schema.
9
10 Key conversions the adapter performs:
11
12 * Line impedances given in **ohms** → per-unit on the system base
13   ( $Z_{base} = kV^2 / S_{base}$ , using the line's voltage level);
14 * a **transformer** becomes a branch too – its ``x_pu`` (on the transformer's own
15   MVA base) is referred to the system base and the transformer is stored as a
16   :class:`Line` flagged ``is_transformer`` (HV side = ``from_bus``);
17 * Loads / generators carry only an ``s_rated_mva`` nameplate in the course files;
18   the **operating point** (P, Q – and hence power factor) is entered by hand, so
19   it defaults to zero on load unless the file also carries ``p_mw`` / ``q_mvar``
20   (which our own saved files do, so edits round-trip). Power factor is never
21   assumed.
22 * Leaf buses (where the operator may pin a voltage) are derived from the
23   topology: a non-slack bus with a single branch.
24
25 A legacy intermediate format (top-level ``buses`` / ``lines`` arrays) is still
26 accepted on read for backwards compatibility.
27
28 Validation is split in two:
29
30 * structural shape / types → raised eagerly while parsing
31 * network-level invariants → :func:`validate_network` (radial tree, single
32   slack, ≤ MAX_BUSES buses, unique ids, dangling references)
33
34 Both surface as :class:`ParserError` so callers have a single thing to catch.
35 """
36 from __future__ import annotations
37
38 import json
39 from collections import defaultdict
40 from pathlib import Path
41
42 from gridlens.core.models import (
43     Bus,
44     Capacitor,
45     Generator,
46     Line,
47     Load,
48     Network,
49 )
50 from gridlens.utils.constants import MAX_BUSES
51
52
53 class ParserError(ValueError):
54     """Raised when a network file is malformed or violates a constraint."""
55
56
57 # ----- #
58 # Public API
59 # ----- #
60 def load_network(path: str | Path) -> Network:
61     """Load and validate a network from a JSON file."""
62     path = Path(path)
63     if not path.exists():
64         raise ParserError(f"File not found: {path}")
65     try:
66         data = json.loads(path.read_text(encoding="utf-8"))
67     except json.JSONDecodeError as e:
68         raise ParserError(f"Invalid JSON in {path.name}: {e}") from e
69     if not isinstance(data, dict):
70         raise ParserError("Top-level network data must be a JSON object.")
```

```

71     if _is_instructor_format(data):
72         network = _from_instructor_dict(data)
73     else:
74         network = _from_dict(data)
75     validate_network(network)
76     return network
77
78
79 def save_network(network: Network, path: str | Path) -> None:
80     """Serialize a network to JSON in the course file format. The output
81     round-trips through :func:`Load_network` without loss."""
82     path = Path(path)
83     text = json.dumps(_to_instructor_dict(network), indent=2, ensure_ascii=False)
84     path.write_text(text + "\n", encoding="utf-8")
85
86
87 def validate_network(network: Network) -> None:
88     """Check the network-level invariants from data/FORMAT.md.
89
90     Raises :class:`ParserError` on the first violation found.
91     """
92     buses = network.buses
93     if not buses:
94         raise ParserError("Network has no buses.")
95     if len(buses) > MAX_BUSES:
96         raise ParserError(
97             f"Network has {len(buses)} buses; the maximum supported is {MAX_BUSES}."
98         )
99
100     bus_ids = [b.id for b in buses]
101     _ensure_unique(bus_ids, "bus")
102     bus_id_set = set(bus_ids)
103
104     # Lines and transformers share the branch namespace (both live in
105     # `network.lines`); their ids must be unique together.
106     _ensure_unique([ln.id for ln in network.lines], "branch")
107     _ensure_unique([x.id for x in network.loads], "load")
108     _ensure_unique([x.id for x in network.generators], "generator")
109     _ensure_unique([x.id for x in network.capacitors], "capacitor")
110
111     # Exactly one slack/source bus.
112     slacks = [b.id for b in buses if b.is_slack]
113     if len(slacks) == 0:
114         raise ParserError("No slack bus defined (exactly one bus needs is_slack: true).")
115     if len(slacks) > 1:
116         raise ParserError(f"Multiple slack buses defined: {' '.join(slacks)}.")
117
118     # Pinned voltage only makes sense on leaf buses.
119     for b in buses:
120         if b.v_set_pu is not None and not b.is_leaf:
121             raise ParserError(
122                 f"Bus '{b.id}' has v_set_pu but is not a leaf (is_leaf: false).")
123
124
125     # Every reference must resolve to a real bus.
126     for ln in network.lines:
127         for endpoint in (ln.from_bus, ln.to_bus):
128             if endpoint not in bus_id_set:
129                 kind = "Transformer" if ln.is_transformer else "Line"
130                 raise ParserError(
131                     f"{kind} '{ln.id}' references unknown bus '{endpoint}'."
132                 )
133         if ln.from_bus == ln.to_bus:
134             raise ParserError(f"Branch '{ln.id}' connects bus '{ln.from_bus}' to itself.")
135
136     for kind, items in (
137         ("load", network.loads),
138         ("generator", network.generators),
139         ("capacitor", network.capacitors),
140     ):
141         for item in items:
142             if item.bus not in bus_id_set:
143                 raise ParserError(

```

```

144         f"{kind.capitalize()} '{item.id}' references unknown bus '{item.bus}'."
145     )
146
147     _ensure_radial_tree(network, bus_id_set)
148
149
150 # ----- #
151 # Instructor (course) format
152 # ----- #
153 def _is_instructor_format(data: dict) -> bool:
154     return any(
155         k in data
156         for k in ("system_data", "bus_data", "line_data", "transformer_data")
157     )
158
159
160 def _from_instructor_dict(data: dict) -> Network:
161     system = _seq(data, "system_data")
162     sys0 = _as_obj(system[0], "system_data[0]") if system else {}
163     name = _opt_str(sys0, "network_name", "") or ""
164     base_mva = _opt_num(sys0, "s_base_mva", 1.0)
165     slack_id = _opt_id(sys0, "slack_bus")
166
167     network = Network(name=name, base_mva=base_mva)
168
169     bus_kv: dict[str, float] = {}
170     for i, d in enumerate(_seq(data, "bus_data")):
171         ctx = f"bus #{i + 1}"
172         obj = _as_obj(d, ctx)
173         bid = _req_id(obj, "bus_id", ctx)
174         kv = _opt_num(obj, "voltage_level_kv", 1.0)
175         bus_kv[bid] = kv
176         network.buses.append(
177             Bus(
178                 id=bid,
179                 name=_opt_str(obj, "bus_name", ""),
180                 base_kv=kv,
181                 is_slack=(slack_id is not None and bid == slack_id),
182                 is_leaf=False, # derived from topology below
183                 v_set_pu=_opt_num_or_none(obj, "v_set_pu"),
184             )
185         )
186
187     # Lines: impedance given in ohms, referred to per-unit on the system base.
188     for i, d in enumerate(_seq(data, "line_data")):
189         ctx = f"line #{i + 1}"
190         obj = _as_obj(d, ctx)
191         fb = _req_id(obj, "from_bus_id", ctx)
192         tb = _req_id(obj, "to_bus_id", ctx)
193         z_base = _z_base(bus_kv.get(fb, 1.0), base_mva)
194         network.lines.append(
195             Line(
196                 id=_req_id(obj, "line_id", ctx),
197                 from_bus=fb,
198                 to_bus=tb,
199                 r_pu=_opt_num(obj, "r_ohm", 0.0) / z_base,
200                 x_pu=_opt_num(obj, "x_ohm", 0.0) / z_base,
201                 b_pu=0.0,
202             )
203         )
204
205     # Transformers: a branch with its reactance referred to the system base. The
206     # internal id is "T"-prefixed so it never collides with a line id.
207     for i, d in enumerate(_seq(data, "transformer_data")):
208         ctx = f"transformer #{i + 1}"
209         obj = _as_obj(d, ctx)
210         hv = _req_id(obj, "hv_bus_id", ctx)
211         lv = _req_id(obj, "lv_bus_id", ctx)
212         rated = _opt_num(obj, "rated_s_mva", base_mva)
213         x_own = _opt_num(obj, "x_pu", 0.0)
214         x_sys = x_own * (base_mva / rated) if rated else x_own
215         network.lines.append(
216             Line(

```

```

217         id=_req_id(obj, "transformer_id", ctx),
218         from_bus=hv,
219         to_bus=lv,
220         r_pu=0.0,
221         x_pu=x_sys,
222         b_pu=0.0,
223         is_transformer=True,
224         xfmr_rated_mva=rated,
225         xfmr_hv_kv=_opt_num_or_none(obj, "v_rated_high_kv"),
226         xfmr_lv_kv=_opt_num_or_none(obj, "v_rated_low_kv"),
227         xfmr_x_pu=x_own,
228     )
229 )
230
231 for i, d in enumerate(_seq(data, "load_data")):
232     ctx = f"load #{i + 1}"
233     obj = _as_obj(d, ctx)
234     network.loads.append(
235         Load(
236             id=_req_id(obj, "load_id", ctx),
237             bus=_req_id(obj, "bus_id", ctx),
238             p_kw=_opt_num(obj, "p_mw", 0.0) * 1000.0,
239             q_kvar=_opt_num(obj, "q_mvar", 0.0) * 1000.0,
240             s_rated_mva=_opt_num(obj, "s_rated_mva", 0.0),
241         )
242     )
243
244 for i, d in enumerate(_seq(data, "gen_data")):
245     ctx = f"generator #{i + 1}"
246     obj = _as_obj(d, ctx)
247     network.generators.append(
248         Generator(
249             # Spec uses `gen_id`; the course's own example file uses
250             # `generator_id` – accept either so both parse.
251             id=_req_id_any(obj, ("gen_id", "generator_id"), ctx),
252             bus=_req_id(obj, "bus_id", ctx),
253             p_kw=_opt_num(obj, "p_mw", 0.0) * 1000.0,
254             q_kvar=_opt_num(obj, "q_mvar", 0.0) * 1000.0,
255             s_rated_mva=_opt_num(obj, "s_rated_mva", 0.0),
256         )
257     )
258
259 for i, d in enumerate(_seq(data, "shunt_data")):
260     ctx = f"shunt #{i + 1}"
261     obj = _as_obj(d, ctx)
262     # Course sign convention: q_mvar > 0 absorbs reactive power (a reactor),
263     # q_mvar < 0 supplies it (a capacitor). Our internal Capacitor.q_kvar is
264     # the *supplied* reactive (B>0 capacitive), so flip the sign.
265     network.capacitors.append(
266         Capacitor(
267             id=_req_id(obj, "shunt_id", ctx),
268             bus=_req_id(obj, "bus_id", ctx),
269             q_kvar=-_opt_num(obj, "q_mvar", 0.0) * 1000.0,
270             in_service=_opt_bool(obj, "in_service", True),
271         )
272     )
273
274 _mark_leaves(network)
275 return network
276
277
278 def _to_instructor_dict(network: Network) -> dict:
279     slack = next((b.id for b in network.buses if b.is_slack), None)
280     system: dict = {"network_name": network.name, "s_base_mva": network.base_mva}
281     if slack is not None:
282         system["slack_bus"] = _id_out(slack)
283
284     bus_kv = {b.id: b.base_kv for b in network.buses}
285     bus_data = []
286     for b in network.buses:
287         entry: dict = {
288             "bus_id": _id_out(b.id),
289             "bus_name": b.name,

```

```

290         "voltage_level_kv": b.base_kv,
291     }
292     if b.v_set_pu is not None:
293         entry["v_set_pu"] = b.v_set_pu
294     bus_data.append(entry)
295
296 line_data = []
297 transformer_data = []
298 for ln in network.lines:
299     if ln.is_transformer:
300         transformer_data.append(
301             {
302                 "transformer_id": _id_out(_strip_t(ln.id)),
303                 "hv_bus_id": _id_out(ln.from_bus),
304                 "lv_bus_id": _id_out(ln.to_bus),
305                 "v_rated_high_kv": ln.xfmr_hv_kv
306                 if ln.xfmr_hv_kv is not None
307                 else bus_kv.get(ln.from_bus),
308                 "v_rated_low_kv": ln.xfmr_lv_kv
309                 if ln.xfmr_lv_kv is not None
310                 else bus_kv.get(ln.to_bus),
311                 "rated_s_mva": ln.xfmr_rated_mva
312                 if ln.xfmr_rated_mva is not None
313                 else network.base_mva,
314                 "x_pu": ln.xfmr_x_pu if ln.xfmr_x_pu is not None else ln.x_pu,
315             }
316         )
317     else:
318         z_base = _z_base(bus_kv.get(ln.from_bus, 1.0), network.base_mva)
319         line_data.append(
320             {
321                 "line_id": _id_out(ln.id),
322                 "from_bus_id": _id_out(ln.from_bus),
323                 "to_bus_id": _id_out(ln.to_bus),
324                 "r_ohm": ln.r_pu * z_base,
325                 "x_ohm": ln.x_pu * z_base,
326             }
327         )
328
329 load_data = [
330     {
331         "load_id": _id_out(x.id),
332         "bus_id": _id_out(x.bus),
333         "s_rated_mva": x.s_rated_mva,
334         "p_mw": x.p_kw / 1000.0,
335         "q_mvar": x.q_kvar / 1000.0,
336     }
337     for x in network.loads
338 ]
339 gen_data = [
340     {
341         "gen_id": _id_out(x.id),
342         "bus_id": _id_out(x.bus),
343         "s_rated_mva": x.s_rated_mva,
344         "p_mw": x.p_kw / 1000.0,
345         "q_mvar": x.q_kvar / 1000.0,
346     }
347     for x in network.generators
348 ]
349 shunt_data = [
350     {
351         "shunt_id": _id_out(x.id),
352         "bus_id": _id_out(x.bus),
353         "q_mvar": -x.q_kvar / 1000.0, # supplied (internal) → absorbed (course)
354         "in_service": x.in_service,
355     }
356     for x in network.capacitors
357 ]
358
359 return {
360     "system_data": [system],
361     "bus_data": bus_data,
362     "load_data": load_data,

```



```

363         "gen_data": gen_data,
364         "shunt_data": shunt_data,
365         "line_data": line_data,
366         "transformer_data": transformer_data,
367     }
368
369
370 def _z_base(kv: float, base_mva: float) -> float:
371     z = (kv * kv) / base_mva if base_mva else 1.0
372     return z if z else 1.0
373
374
375 def _mark_leaves(network: Network) -> None:
376     """A leaf is a non-slack bus with exactly one incident branch."""
377     degree: dict[str, int] = defaultdict(int)
378     for ln in network.lines:
379         degree[ln.from_bus] += 1
380         degree[ln.to_bus] += 1
381     for b in network.buses:
382         if not b.is_slack and degree[b.id] == 1:
383             b.is_leaf = True
384
385
386 def _strip_t(branch_id: str) -> str:
387     return branch_id[1:] if branch_id.startswith("T") else branch_id
388
389
390 # ----- #
391 # Legacy intermediate format (top-level buses / lines) - read only
392 # ----- #
393 def _from_dict(data: dict) -> Network:
394     network = Network(
395         name=_opt_str(data, "name", "") or "",
396         base_mva=_opt_num(data, "base_mva", 1.0),
397     )
398     network.buses = [_bus_from_dict(d, i) for i, d in enumerate(_seq(data, "buses"))]
399     network.lines = [_line_from_dict(d, i) for i, d in enumerate(_seq(data, "lines"))]
400     network.loads = [
401         _injection_from_dict(Load, "load", d, i)
402         for i, d in enumerate(_seq(data, "loads"))
403     ]
404     network.generators = [
405         _injection_from_dict(Generator, "generator", d, i)
406         for i, d in enumerate(_seq(data, "generators"))
407     ]
408     network.capacitors = [
409         _capacitor_from_dict(d, i) for i, d in enumerate(_seq(data, "capacitors"))
410     ]
411     return network
412
413
414 def _bus_from_dict(d: object, index: int) -> Bus:
415     ctx = f"bus #{index + 1}"
416     obj = _as_obj(d, ctx)
417     return Bus(
418         id=_req_str(obj, "id", ctx),
419         name=_opt_str(obj, "name", ""),
420         base_kv=_opt_num(obj, "base_kv", 1.0),
421         is_slack=_opt_bool(obj, "is_slack", False),
422         is_leaf=_opt_bool(obj, "is_leaf", False),
423         v_set_pu=_opt_num_or_none(obj, "v_set_pu"),
424     )
425
426
427 def _line_from_dict(d: object, index: int) -> Line:
428     ctx = f"line #{index + 1}"
429     obj = _as_obj(d, ctx)
430     return Line(
431         id=_req_str(obj, "id", ctx),
432         from_bus=_req_str(obj, "from_bus", ctx),
433         to_bus=_req_str(obj, "to_bus", ctx),
434         r_pu=_opt_num(obj, "r_pu", 0.0),
435         x_pu=_opt_num(obj, "x_pu", 0.0),

```

```

436         b_pu=_opt_num(obj, "b_pu", 0.0),
437         rating_a=_opt_num_or_none(obj, "rating_a"),
438     )
439
440
441 def _injection_from_dict(cls, kind: str, d: object, index: int):
442     ctx = f"{kind} #{index + 1}"
443     obj = _as_obj(d, ctx)
444     return cls(
445         id=_req_str(obj, "id", ctx),
446         bus=_req_str(obj, "bus", ctx),
447         p_kw=_opt_num(obj, "p_kw", 0.0),
448         q_kvar=_opt_num(obj, "q_kvar", 0.0),
449     )
450
451
452 def _capacitor_from_dict(d: object, index: int) -> Capacitor:
453     ctx = f"capacitor #{index + 1}"
454     obj = _as_obj(d, ctx)
455     return Capacitor(
456         id=_req_str(obj, "id", ctx),
457         bus=_req_str(obj, "bus", ctx),
458         q_kvar=_opt_num(obj, "q_kvar", 0.0),
459         in_service=_opt_bool(obj, "in_service", True),
460     )
461
462
463 # ----- #
464 # Field helpers
465 # ----- #
466
467 def _seq(data: dict, key: str) -> list:
468     value = data.get(key, [])
469     if value is None:
470         return []
471     if not isinstance(value, list):
472         raise ParserError(f"'{key}' must be a list.")
473     return value
474
475
476 def _as_obj(d: object, ctx: str) -> dict:
477     if not isinstance(d, dict):
478         raise ParserError(f"{ctx} must be a JSON object.")
479     return d
480
481
482 def _req_str(obj: dict, key: str, ctx: str) -> str:
483     if key not in obj:
484         raise ParserError(f"{ctx} is missing required field '{key}'")
485     value = obj[key]
486     if not isinstance(value, str) or not value:
487         raise ParserError(f"{ctx} field '{key}' must be a non-empty string.")
488     return value
489
490
491 def _req_id(obj: dict, key: str, ctx: str) -> str:
492     """An id field that may be given as an integer or a string; normalised to str."""
493     if key not in obj or obj[key] is None:
494         raise ParserError(f"{ctx} is missing required field '{key}'")
495     return _id_str(obj[key], key, ctx)
496
497
498 def _req_id_any(obj: dict, keys: tuple[str, ...], ctx: str) -> str:
499     """Like :func:`_req_id` but accepts the first of several alias field names."""
500     for key in keys:
501         if key in obj and obj[key] is not None:
502             return _id_str(obj[key], key, ctx)
503     raise ParserError(f"{ctx} is missing required field '{keys[0]}'")
504
505
506 def _opt_id(obj: dict, key: str) -> str | None:
507     if key not in obj or obj[key] is None:
508         return None
509     return _id_str(obj[key], key, key)

```

```

509
510
511 def _id_str(value: object, key: str, ctx: str) -> str:
512     if isinstance(value, bool):
513         raise ParserError(f"{ctx} field '{key}' must be an id, got {value!r}.")
514     if isinstance(value, int):
515         return str(value)
516     if isinstance(value, str) and value:
517         return value
518     raise ParserError(f"{ctx} field '{key}' must be a non-empty id (string or integer).")
519
520
521 def _id_out(value: str):
522     """Emit numeric ids as integers (matching the course format), others as-is."""
523     if isinstance(value, str) and value.lstrip("-").isdigit():
524         return int(value)
525     return value
526
527
528 def _opt_str(obj: dict, key: str, default: str) -> str:
529     value = obj.get(key, default)
530     if value is None:
531         return default
532     if not isinstance(value, str):
533         raise ParserError(f"Field '{key}' must be a string.")
534     return value
535
536
537 def _opt_num(obj: dict, key: str, default: float) -> float:
538     if key not in obj or obj[key] is None:
539         return default
540     return _coerce_num(obj[key], key)
541
542
543 def _opt_num_or_none(obj: dict, key: str) -> float | None:
544     if key not in obj or obj[key] is None:
545         return None
546     return _coerce_num(obj[key], key)
547
548
549 def _coerce_num(value: object, key: str) -> float:
550     # bool is a subclass of int - reject it so True doesn't silently become 1.0.
551     if isinstance(value, bool) or not isinstance(value, (int, float)):
552         raise ParserError(f"Field '{key}' must be a number, got {value!r}.")
553     return float(value)
554
555
556 def _opt_bool(obj: dict, key: str, default: bool) -> bool:
557     value = obj.get(key, default)
558     if value is None:
559         return default
560     if not isinstance(value, bool):
561         raise ParserError(f"Field '{key}' must be a boolean (true/false).")
562     return value
563
564
565 def _ensure_unique(ids: list[str], kind: str) -> None:
566     seen: set[str] = set()
567     for i in ids:
568         if i in seen:
569             raise ParserError(f"Duplicate {kind} id: '{i}'.")
570     seen.add(i)
571
572
573 def _ensure_radial_tree(network: Network, bus_ids: set[str]) -> None:
574     """A valid radial feeder is a connected, acyclic, undirected graph: it must
575     have exactly (buses - 1) branches (lines + transformers) and be fully
576     connected from the slack bus."""
577     n_buses = len(bus_ids)
578     n_branches = len(network.lines)
579     if n_branches != n_buses - 1:
580         raise ParserError(
581             f"Network is not radial: {n_buses} buses need exactly "

```

```

582         f"{n_buses - 1} branches, but found {n_branches} "
583         "(a radial feeder is a tree)."
584     )
585
586     adjacency: dict[str, list[str]] = {b: [] for b in bus_ids}
587     for ln in network.lines:
588         adjacency[ln.from_bus].append(ln.to_bus)
589         adjacency[ln.to_bus].append(ln.from_bus)
590
591     root = next(b.id for b in network.buses if b.is_slack)
592     visited: set[str] = set()
593     stack = [root]
594     while stack:
595         node = stack.pop()
596         if node in visited:
597             continue
598         visited.add(node)
599         stack.extend(adjacency[node])
600
601     if len(visited) != n_buses:
602         unreachable = sorted(bus_ids - visited)
603         raise ParserError(
604             "Network is not connected (radial feeder must be a single tree); "
605             f"buses unreachable from slack '{root}': {' '.join(unreachable)}."
606         )

```

src/gridlens/core/solver.py

```
1  """VA (power-summation backward-forward sweep) power-flow solver for radial
2  distribution networks.
3
4  The "VA approach" is the volt-ampere (power) summation variant of the
5  backward-forward sweep: the backward pass accumulates the complex *power* flowing
6  in each branch rather than the current. The radial topology lets us avoid
7  building/inverting a Y-bus – we order buses by depth from the slack root, then
8  alternate two sweeps until the bus voltages stop moving.
9
10  1. Order buses by depth from the slack root (BFS traversal of the radial tree).
11  2. **Backward sweep (VA / power summation)** – from leaves toward root,
12     accumulate the complex power (volt-amperes) flowing into each branch:
13      $S_{\text{branch}} = S_{\text{drawn}} + \sum \text{children} (S_{\text{child}} + Z \cdot |S_{\text{child}}|^2 / |V_{\text{child}}|^2)$ 
14     where  $S_{\text{drawn}} = (P_{\text{load}} - P_{\text{gen}}) + j(Q_{\text{load}} - Q_{\text{gen}}) - jB_{\text{shunt}} \cdot |V|^2$ 
15     (capacitors / line charging enter as shunt susceptance,  $B > 0$  capacitive →
16     supplies reactive power) and the  $Z \cdot |S|^2 / |V|^2$  term is the branch series loss
17     (power, unlike current, is not conserved across a branch impedance).
18  3. **Forward sweep** – from root toward leaves, recover the branch current from
19     its VA flow and update voltages:
20      $I_{\text{branch}} = \text{conj}(S_{\text{branch}} / V_{\text{to}})$ ;  $V_{\text{to}} = V_{\text{from}} - Z_{\text{branch}} \cdot I_{\text{branch}}$ 
21  4. Repeat until  $\max |V_k - V_{\{k-1\}}| < \text{tol}$  or  $\text{iter} > \text{max\_iter}$ .
22
23  Current and power summation are algebraically equivalent for a radial feeder
24  ( $I = (S/V)^*$ ), so this converges to the same bus voltages and angles; the VA
25  form is used to match the project's required solver approach.
26
27  Operator-pinned leaf voltage (Bus.v_set_pu):
28  Plain BFS converges with PQ buses + a single slack at a known voltage. When the
29  operator instead pins a *leaf* voltage, the boundary condition flips: the leaf
30  voltage is the known quantity and the slack/grid voltage becomes the unknown.
31  An outer loop solves for it:
32  a) Run the inner sweep with the current slack-voltage guess (all PQ).
33  b) Read the resulting pinned-leaf  $|V|$  and compute  $\Delta V = V_{\text{set}} - |V|$ .
34  c) Scale the slack voltage toward the set-point:  $V_{\text{slack}} := V_{\text{set}} / |V_{\text{leaf}}|$ .
35  d) Repeat until  $|\Delta V| < \text{tol}$ .
36  No reactive power is injected – holding the leaf simply determines what the
37  source voltage must be (the slack voltage changes; everything else follows).
38  Generators in this model are pure PQ injections (no regulated  $|V|$ ).
39
40  Capacitor banks are modeled as shunt susceptances toggled by `in_service`.
41  """
42  from __future__ import annotations
43
44  import cmath
45  import math
46  from collections import defaultdict, deque
47
48  from gridlens.core.models import BusSolution, Network, SolutionResult, SweepStep
49  from gridlens.utils.constants import (
50      DEFAULT_MAX_ITER,
51      DEFAULT_TOLERANCE_PU,
52      V_MAX_PU,
53      V_MIN_PU,
54  )
55
56  _MAX_OUTER_ITER = 40
57  _OUTER_DAMPING = 0.8
58
59  # Divergence guards: a radial sweep that runs away (a bus voltage collapsing to
60  # zero or blowing up) means the operating point is beyond the feeder's
61  # loadability. Detect it and report non-convergence instead of raising.
62  _MIN_VOLTAGE = 1e-9
63  _MAX_VOLTAGE = 1e3 # any per-unit voltage this large means the sweep is running away
64
65
66  def solve(
67      network: Network,
68      *,
69      tol: float = DEFAULT_TOLERANCE_PU,
70      max_iter: int = DEFAULT_MAX_ITER,
```

```

71 ) -> SolutionResult:
72     """Run BFS power flow on a radial network. Returns voltage magnitude/angle per bus."""
73     if not network.buses:
74         return SolutionResult(
75             converged=False, iterations=0, max_mismatch=0.0,
76             message="Network has no buses.",
77         )
78
79     slack = next((b for b in network.buses if b.is_slack), None)
80     if slack is None:
81         return SolutionResult(
82             converged=False, iterations=0, max_mismatch=0.0,
83             message="No slack bus defined.",
84         )
85
86     topo = _build_topology(network, slack.id)
87     if topo is None:
88         return SolutionResult(
89             converged=False, iterations=0, max_mismatch=0.0,
90             message="Network is not a connected radial tree.",
91         )
92     order, parent, children, branch_z = topo
93
94     s_base_kw = network.base_mva * 1000.0
95     p_net, q_load_net, b_shunt = _node_injections(network, s_base_kw)
96     v_slack = complex(slack.v_set_pu if slack.v_set_pu is not None else 1.0, 0.0)
97
98     pinned = {
99         b.id: b.v_set_pu
100         for b in network.buses
101         if b.v_set_pu is not None and not b.is_slack
102     }
103
104     # Outer loop. When a leaf voltage is pinned, the operator fixes that leaf and
105     # the slack/grid voltage becomes the unknown: we hold the pinned leaf at its
106     # set-point and let the slack voltage take whatever value the radial drop
107     # requires. (Per the course clarification – "fix the leaf, the source voltage
108     # changes" – no reactive power is injected to do this.) With no pinned leaf
109     # this runs exactly once with the slack fixed at its nominal voltage.
110     v_mag = abs(v_slack) # slack voltage magnitude (solved for if pinned)
111     ctrl_bus = next(iter(pinned), None) # the leaf whose set-point drives the slack
112     voltages: dict[str, complex] = {}
113     inner_iters = 0
114     inner_mismatch = 0.0
115     inner_ok = False
116     inner_reason = ""
117     outer_ok = True
118     mismatch_hist = []
119     steps: list[SweepStep] = []
120     outer_idx = 0
121     # ``max_iter`` is a *total* sweep budget shared across every outer pass, so the
122     # iteration count the user sees never exceeds the configured limit even when a
123     # pinned leaf needs several slack-voltage adjustment passes.
124     budget = max_iter
125
126     for _ in range(_MAX_OUTER_ITER):
127         if budget <= 0:
128             outer_ok = False
129             break
130         outer_idx += 1
131         voltages, inner_iters, inner_mismatch, inner_ok, inner_reason, mismatch_hist = _converge_inner(
132             order, parent, children, branch_z,
133             p_net, q_load_net, b_shunt, complex(v_mag, 0.0), tol, budget,
134         )
135         budget -= inner_iters
136         # Record this pass's inner sweep as part of the full trajectory.
137         for j, dv_inner in enumerate(mismatch_hist, start=1):
138             steps.append(SweepStep(outer=outer_idx, inner=j, max_dv=dv_inner))
139         if not inner_ok:
140             outer_ok = False
141             break
142         if not pinned:
143             break

```

```

144
145     # Hold the pinned leaf by scaling the slack voltage toward the set-point.
146     pinned_err = max(abs(v_set - abs(voltages[bus])) for bus, v_set in pinned.items())
147     if steps:
148         steps[-1].pinned_err = pinned_err
149     if pinned_err < tol:
150         outer_ok = True
151         break
152     outer_ok = False
153     v_leaf = abs(voltages[ctrl_bus])
154     if v_leaf > _MIN_VOLTAGE:
155         ratio = pinned[ctrl_bus] / v_leaf
156         v_mag *= 1.0 + _OUTER_DAMPING * (ratio - 1.0)
157
158     bus_results = [
159         _bus_solution(b.id, voltages.get(b.id, complex(float("nan"))))
160         for b in network.buses
161     ]
162     converged = inner_ok and outer_ok
163     if converged:
164         message = "Converged."
165         if pinned:
166             message += (
167                 f" Grid (slack) voltage settled at {v_mag:.4f} pu to hold the "
168                 "pinned leaf at its set-point."
169             )
170     elif not inner_ok:
171         if inner_reason == "diverged":
172             message = (
173                 "Solver diverged – the operating point may be beyond the "
174                 "feeder's loadability (check loads, generation, or the pinned "
175                 "leaf voltage)."
176             )
177         else:
178             message = "Did not converge within the iteration limit."
179     else:
180         message = (
181             "Pinned leaf voltage could not be held within tolerance; showing "
182             "the closest result."
183         )
184     return SolutionResult(
185         converged=converged,
186         iterations=inner_iters,
187         max_mismatch=inner_mismatch,
188         bus_results=bus_results,
189         message=message,
190         mismatch_history=mismatch_hist,
191         steps=steps,
192         outer_iterations=outer_idx,
193         # No reactive power is injected to hold the leaf now (the slack voltage is
194         # adjusted instead), so there is no required-Q figure to report.
195         pinned_q_inject_kvar={},
196     )
197
198
199 # ----- #
200 # Topology
201 # ----- #
202 def _build_topology(network: Network, root: str):
203     """BFS the radial tree from the slack root.
204
205     Returns (order, parent, children, branch_z) where order is breadth-first
206     bus ids, parent[bus]/children[bus] describe the rooted tree, and
207     branch_z[bus] is the series impedance of the line joining bus to its parent.
208     Returns None if the graph is not a single connected tree.
209     """
210     adjacency: dict[str, list[tuple[str, complex]]] = defaultdict(list)
211     for ln in network.lines:
212         z = complex(ln.r_pu, ln.x_pu)
213         adjacency[ln.from_bus].append((ln.to_bus, z))
214         adjacency[ln.to_bus].append((ln.from_bus, z))
215
216     order: list[str] = []

```

```

217     parent: dict[str, str] = {}
218     children: dict[str, list[str]] = defaultdict(list)
219     branch_z: dict[str, complex] = {}
220     visited = {root}
221     queue: deque[str] = deque([root])
222     while queue:
223         u = queue.popleft()
224         order.append(u)
225         for v, z in adjacency[u]:
226             if v not in visited:
227                 visited.add(v)
228                 parent[v] = u
229                 branch_z[v] = z
230                 children[u].append(v)
231                 queue.append(v)
232
233     if len(visited) != len(network.buses):
234         return None
235     return order, parent, children, branch_z
236
237
238 # ----- #
239 # Injections (per-unit, load convention: positive = drawn from network)
240 # ----- #
241 def _node_injections(network: Network, s_base_kw: float):
242     p_net: dict[str, float] = {b.id: 0.0 for b in network.buses}
243     q_net: dict[str, float] = {b.id: 0.0 for b in network.buses}
244     b_shunt: dict[str, float] = {b.id: 0.0 for b in network.buses}
245
246     for load in network.loads:
247         p_net[load.bus] += load.p_kw / s_base_kw
248         q_net[load.bus] += load.q_kvar / s_base_kw
249     for gen in network.generators:
250         p_net[gen.bus] -= gen.p_kw / s_base_kw
251         q_net[gen.bus] -= gen.q_kvar / s_base_kw
252     for cap in network.capacitors:
253         if cap.in_service:
254             # Rated kvar at 1 pu -> susceptance B; injects Q = B/V|^2.
255             b_shunt[cap.bus] += cap.q_kvar / s_base_kw
256     for ln in network.lines:
257         half = ln.b_pu / 2.0
258         b_shunt[ln.from_bus] += half
259         b_shunt[ln.to_bus] += half
260     return p_net, q_net, b_shunt
261
262
263 # ----- #
264 # Inner backward-forward sweep
265 # ----- #
266 # Under-relaxation schedule. A plain sweep (damping 1.0) converges fast for
267 # well-conditioned feeders, but stiff cases – a shunt behind a large series
268 # impedance, as in low-voltage feeders with ohmic data – can make the fixed-point
269 # iteration overshoot and diverge. When that happens we retry with progressively
270 # heavier damping, which shrinks the iteration gain below 1 without moving the
271 # fixed point (the converged values are identical).
272 _DAMPING_SCHEDULE = (1.0, 0.5, 0.25, 0.1, 0.05)
273
274
275 def _converge_inner(
276     order, parent, children, branch_z,
277     p_net, q_net, b_shunt, v_slack, tol, max_iter,
278 ):
279     """Run the inner sweep, falling back to heavier under-relaxation if it does
280     not converge. Returns the first converged result, or the last attempt.
281
282     ``max_iter`` is honoured as a hard per-attempt cap: every damping level gets
283     the same iteration budget the user configured, so the reported iteration
284     count never exceeds it. If a genuinely stiff network needs more, the operator
285     raises Max iterations – the solver does not silently overrun the setting."""
286     result = None
287     for damping in _DAMPING_SCHEDULE:
288         result = _inner_sweep(
289             order, parent, children, branch_z,

```



```

290         p_net, q_net, b_shunt, v_slack, tol, max_iter, damping,
291     )
292     if result[3]: # converged
293         return result
294     return result
295
296
297 def _inner_sweep(
298     order, parent, children, branch_z,
299     p_net, q_net, b_shunt, v_slack, tol, max_iter, damping=1.0,
300 ):
301     slack = order[0]
302     voltages: dict[str, complex] = {bus: complex(abs(v_slack), 0.0) for bus in order}
303     voltages[slack] = v_slack
304
305     converged = False
306     iterations = 0
307     max_dv = 0.0
308     mismatch_history = []
309     for iterations in range(1, max_iter + 1):
310         prev = dict(voltages)
311
312         # Bail out before dividing by a collapsed / runaway voltage.
313         for bus in order:
314             mag = abs(voltages[bus])
315             if not cmath.isfinite(voltages[bus]) or mag < _MIN_VOLTAGE or mag > _MAX_VOLTAGE:
316                 return voltages, iterations, float("inf"), False, "diverged", mismatch_history
317
318         # Backward sweep (VA / power summation): from leaves to root, accumulate
319         # the complex power (volt-amperes) flowing into each branch – the local
320         # drawn power plus every downstream branch power *and its series loss*.
321         # Losses are explicit here because, unlike current, power is not conserved
322         # across a branch impedance.
323         s_branch: dict[str, complex] = {}
324         for bus in reversed(order):
325             v = voltages[bus]
326             # Local power drawn from upstream: Load – generation, minus the
327             # reactive power a shunt (capacitor B>0 / line charging) supplies,
328             #  $S_{shunt} = -jB|V|^2$  (B>0 capacitive → injects Q, lifts the voltage).
329             s_drawn = complex(p_net[bus], q_net[bus]) - 1j * b_shunt[bus] * (abs(v) ** 2)
330             total = s_drawn
331             for ch in children[bus]:
332                 s_ch = s_branch[ch]
333                 loss = branch_z[ch] * (abs(s_ch) ** 2) / (abs(voltages[ch]) ** 2)
334                 total += s_ch + loss
335             s_branch[bus] = total
336
337         # Forward sweep: propagate voltages from root outward, recovering each
338         # branch current from its VA flow,  $I_{branch} = \text{conj}(S_{branch} / V_{to})$ .
339         # Under-relaxation (damping < 1) blends the new estimate with the old to
340         # keep stiff cases stable; it does not change the converged fixed point.
341         for bus in order:
342             if bus == slack:
343                 continue
344             i_branch = (s_branch[bus] / voltages[bus]).conjugate()
345             target = voltages[parent[bus]] - branch_z[bus] * i_branch
346             voltages[bus] = voltages[bus] + damping * (target - voltages[bus])
347
348         max_dv = max(abs(voltages[b] - prev[b]) for b in order)
349         mismatch_history.append(max_dv)
350         if not math.isfinite(max_dv):
351             return voltages, iterations, float("inf"), False, "diverged", mismatch_history
352         if max_dv < tol:
353             converged = True
354             break
355
356     return voltages, iterations, max_dv, converged, (" " if converged else "iterations"), mismatch_history
357
358
359 # ----- #
360 # Result formatting
361 # ----- #
362 def _bus_solution(bus_id: str, v: complex) -> BusSolution:

```

```
363     mag = abs(v)
364     if not math.isfinite(mag):
365         return BusSolution(bus_id=bus_id, v_pu=float("nan"), angle_deg=float("nan"))
366     angle = math.degrees(cmath.phase(v))
367     if mag < V_MIN_PU:
368         violation = "under"
369     elif mag > V_MAX_PU:
370         violation = "over"
371     else:
372         violation = "ok"
373     return BusSolution(bus_id=bus_id, v_pu=mag, angle_deg=angle, violation=violation)
```

src/gridlens/core/__init__.py

```
1  from gridlens.core.models import (
2      Bus,
3      Capacitor,
4      Generator,
5      Line,
6      Load,
7      Network,
8      SolutionResult,
9  )
10 from gridlens.core.parser import (
11     ParserError,
12     load_network,
13     save_network,
14     validate_network,
15 )
16 from gridlens.core.solver import solve
17
18 __all__ = [
19     "Bus",
20     "Capacitor",
21     "Generator",
22     "Line",
23     "Load",
24     "Network",
25     "SolutionResult",
26     "ParserError",
27     "load_network",
28     "save_network",
29     "validate_network",
30     "solve",
31 ]
```